COS30049 Computing Technology Innovation Project

Bitcoin Address Activity Explorer

Assignment 2: Data Processing and Machine Learning

Detecting Unusual Transactions with Machine Learning

Group HCMC_COS30049.1_4 (Group 4)
Tutor Arthur Hoang
Campus Swinburne University of Technology, Ho Chi Minh City
Date November 2026

No	Team member	Student ID
1	Cam Dan Bui	105967074
2	Cao Tri Tran	106214726
3	Le Quang Anh Hoang	106218605
4	Minh Khang Nguyen	106216803

Word count: 2,991 words in the body (Sections 1–8), counted by `tools/report_wordcount.py`; tables, captions, the cover, contents, bibliography and appendix are excluded.

Contents

1 Introduction **1**

2 Problem Framing **1**

3 Data Collection **1**
3.1 Our own collections: DS1, DS2 and DS10 1
3.2 Public labelled data and text corpora 2
3.3 Challenges and how we addressed them 2

4 Data Processing **2**

5 Data Analysis **3**

6 Machine Learning Model Selection **5**

7 Implementation **6**
7.1 Technical implementation 6
7.2 Implementation evaluation 6

8 Conclusion **9**

Bibliography **10**

A Requirements checklist **12**

B System architecture and workflow **12**

C Feature walkthroughs **13**

D Key code **14**

E Example prediction **15**

F Further results **16**

G Reproducing the results **21**

1 Introduction

Banks, exchanges and payment providers that handle cryptocurrency must decide which of many transactions deserve a closer look, the topic the brief calls cybersecurity threat detection in banking. A Bitcoin address's history is public, but a block explorer shows it only as a long, unexplained list.

Our project, the Bitcoin Address Activity Explorer, ranks every transaction of an address by how *unusual* it is *for that address's own past* and says why in plain words. Its intended users are student researchers and compliance or blockchain analysts examining one address who need to know where to look first. It is an investigation aid, never a verdict: every flag carries the line *Unusual activity requires investigation; it does not indicate wrongdoing.*

Our team trained every model, both language models from scratch.

2 Problem Framing

The challenge. We must decide which transactions deserve attention when no labels exist, and explain each decision, because the user acts on the reason, not on a score.

Limits of existing solutions. Block explorers list transactions without ranking them. A fixed rule, such as "flag anything above one bitcoin", ignores scale: wallets, trading bots and cold storage behave very differently, and the median amounts of our addresses differ by a factor of 9,949. One threshold over-flags busy addresses, misses changes on quiet ones, and cannot follow changing behaviour. Supervised models trained on public labelled datasets learn from sparse labels and from aggregated or anonymised features that cannot be computed for an arbitrary address (Akcora et al. 2020; Elmougy and Liu 2023), and their accuracy collapses when behaviour changes (Pareja et al. 2020).

Why machine learning. Learning can model each address's normal behaviour, find structure without labels, read a history as a sequence, and use labels where they exist (Hilal, Gadsden and Yawney 2022; Han et al. 2022). Real transactions have no target label, so we combine unsupervised methods with regression and use classification only on labelled data. Four research questions guide the work:

RQ1 Which stable method best finds transactions planted in an address's own history?
RQ2 How well do classifiers perform on labelled data over time and on unseen families?
RQ3 Does "unusual" coincide with the labelled datasets' own illicit class?
RQ4 Does a sequence model trained on other addresses generalise better than per-address statistics, and can a language model trained from scratch explain flags faithfully?

3 Data Collection

Table 1 lists the eight datasets.

3.1 Our own collections: DS1, DS2 and DS10

We use Blockstream's Esplora API (Blockstream n.d.) because it returns each address's confirmed history with every transaction's inputs and outputs. The DS1 selection rule was recorded, with its random seed, before any address was fetched: take 20 block heights from the previous four weeks, gather their output addresses (5,995 candidates), shuffle them and visit them in order. A *main* address needs 50–1,000 confirmed transactions over at least 90 days; a *short-history* address needs 3–9, to test the "not enough history" state. An address is kept only if its transaction count and its received and sent sums match the API's totals, and no transaction is duplicated or unconfirmed. DS1 holds 25 main and 3 short addresses with 7,966 transactions.

DS2 applies the same client to 200 blocks spread over twelve months, after removing every address in DS1's candidate pool (1,533) so that DS1 never leaks into TxGPT's training data. Of 9,079 addresses visited, 4,012 were kept, stored only as salted hashes. DS10 is a teammate's independent snapshot with

Table 1: The datasets and the role of each (tx: transactions).

ID	Dataset	Source (licence)	Size	Role
DS1	Our Esplora snapshot	Blockstream Esplora API (public data)	25 + 3 addresses, 7,966 tx	Every scoring method; evaluation
DS2	Our Esplora crawl (hashed IDs)	Esplora API via mempool.space	4,012 addresses, 898,490 tx	Trains TxGPT; check on unseen addresses
DS3	Elliptic data set	Kaggle (CC BY-NC-ND 4.0)	203,769 tx	Classification
DS4	BitcoinHeist	UCI repository 526 (CC BY 4.0)	2,916,697 address-days	Classification
DS5	FineWeb-Edu	Hugging Face (ODC-By 1.0)	5.0 billion tokens used	Assistant pretraining
DS6	smol-smoltalk	Hugging Face (Apache 2.0)	chat conversations	Assistant chat stage
DS7	Our explanation pairs	Built from DS1 and DS2 facts	question–answer pairs	Teaching and testing the Assistant
DS10	A teammate's snapshot	Esplora API, refetched and cross-checked	27 addresses, 5,785 tx	Held-out final test of every model

its own script and rule: 6 random recent blocks, 8 addresses in each of three activity tiers so that busy addresses do not dominate, and 3 short histories. Refetched with our client, all 5,785 transactions agree, and no address is shared with DS1 or DS2.

3.2 Public labelled data and text corpora

Elliptic (DS3) labels 46,564 of its 203,769 transactions as illicit or licit, its own class names, over 49 time steps (Elliptic 2019; Elmougy and Liu 2023). BitcoinHeist (DS4) has 2,916,697 address-days, of which 41,413 belong to ransomware families (Akcora et al. 2020; UCI Machine Learning Repository 2020). Their labels allow the accuracy, precision, recall and F1 that real addresses cannot support. Elliptic's licence allows no derivatives, so we ship a download script instead. The Assistant learned English from FineWeb-Edu (DS5; Penedo et al. 2024), conversation from smol-smoltalk (DS6; Ben Allal et al. 2025), and explanations from DS7, question–answer pairs built from our own pipeline's facts.

3.3 Challenges and how we addressed them

- **Rate limits.** The public API allows about 700 requests an hour. Our client paces requests, backs off on refusals, falls back to the compatible mempool.space API (mempool.space n.d.), and caches every reply; 106 addresses that failed every retry were logged and skipped.
- **Selection bias.** 539 visited addresses exceeded the transaction cap (typically busy services) and 4,422 had too little history. Sampling recent blocks favours active addresses.
- **No ground truth.** We evaluate on planted transactions (Section 4) and use labelled metrics only on DS3 and DS4.

4 Data Processing

Figure 1 shows the pipeline; each step is one module of our `explorer` package.

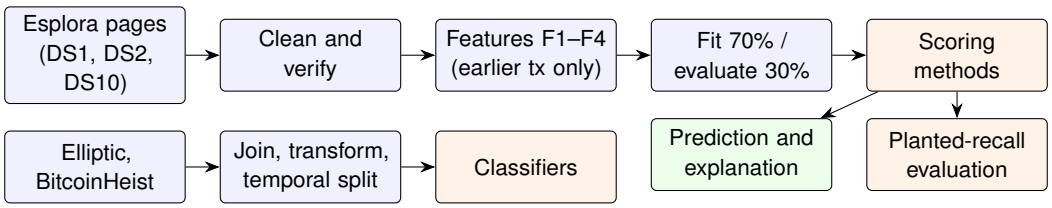


Figure 1: The processing pipeline. Our Bitcoin data (top) feeds the scoring methods; the labelled datasets (bottom) feed the classifiers. The two paths never mix.

Cleaning and missing values. DS1 passes five checks: parse every saved page, keep confirmed transactions, remove duplicate (address, transaction) pairs, drop transactions that neither pay nor spend from the address, and sort by time. None removed a row, because collection had already rejected faulty histories. The source data has no missing values; features are empty by design when an address has too little history, and such transactions are shown as "not scored", because imputing a history could create or hide a flag. Outliers are kept, never removed or capped: they are what the Explorer looks for.

Feature engineering. For each transaction we compute the address's net amount (received minus sent) and direction, then the four features of Table 2. Each is compared with the same address's history using a robust z-score: the distance from the median in units of the median absolute deviation, which one extreme past value cannot distort (Blázquez-García et al. 2022). Heavy-tailed features are compared on a logarithmic scale, and the spread has a floor so that a run of identical values cannot divide by zero. This is the normalisation: every score is relative to the address itself. Distance-based models also standardise their inputs, so that no feature dominates by scale.

Table 2: The four explained features (A1, refined). Each becomes a robust z-score against the same address's earlier history.

ID	Name	Definition	Compared on
F1	Size	Amount the address received or sent (BTC)	log10; same direction only
F2	Burst	Earlier transactions in the previous 7 days	raw count
F3	Gap	Hours since the address's previous transaction	log10
F4	Relative size	Amount divided by the usual amount in the same direction	log10

Each feature is compared with earlier transactions of the same address in the previous 30 days, extended back to reach 20; a transaction is scored once the address has at least 10 earlier transactions over 14 days.

Time safety. A transaction's features use only *strictly earlier* transactions of the same address, and transactions in the same block are never each other's history. Models fit on each address's earlier 70% and are evaluated on the rest, and every scaler sits inside a scikit-learn pipeline fitted on training rows only. A random 80:20 split would let a model learn from an address's future, so these rules replace it to prevent leakage (Kapoor and Narayanan 2023).

Planted transactions. To measure detection without labels we plant three patterns into copies of each history's evaluation period (Lai et al. 2021): a *spike* of 10× the address's usual amount, a *burst* of 10 transactions within 24 hours, and a *dormant* address waking after 60–120 days. Each seed plants 50 of each pattern.

Labelled datasets. Elliptic's features and classes are joined on the transaction identifier and split by time (steps 1–34 to train and validate, 35–49 to test), with all features (AF) or local ones (LF). In BitcoinHeist every ransomware family becomes the positive class, the five heavy-tailed features are log transformed, and `year`, `day` and `address` are excluded as features because they would leak the split or identify the row.

Language-model inputs. TxGPT writes each transaction as six tokens (gap, direction, amount, inputs, outputs, fee rate) from a vocabulary of 94.

5 Data Analysis

The analysis is in `notebooks/01_eda.ipynb`, with every statistic in `eda_tests.csv`. Each finding gives the observation, its statistic and the decision it informed.

Finding 1: the features are heavy-tailed. Amounts, gaps and ratios span several orders of magnitude (Figure 2). The raw amount has a skewness of 38.9, but only −0.44 on a logarithmic scale; the size ratio's excess kurtosis is 7,813. A mean and standard deviation would be dominated by a few extreme values, so we compare on log scales and use the median and median absolute deviation.

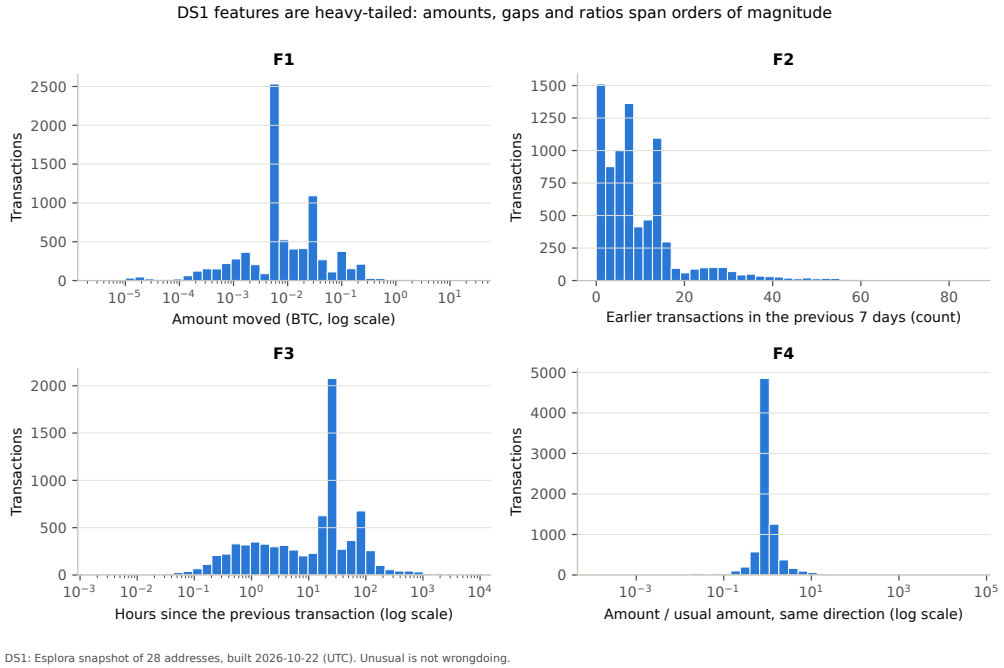


Figure 2: DS1's four features. Amounts (F1), gaps (F3) and size ratios (F4) span orders of magnitude, which is why they are compared on log scales with robust statistics.

Finding 2: every address has its own scale. The median amounts of DS1's main addresses differ by a factor of 9,949, and for 9 of the 23 addresses with enough transactions in both directions, received and sent amounts differ (Mann–Whitney U, $p < 0.05$; Figure 3). One global threshold cannot fit every address, so each feature is compared with the same address's history and amounts only with earlier transactions in the same direction. Ablation confirms this: A1's original definition flags 31.3% of real transactions with mean planted recall 0.439, against 10.4% and 0.655 for the refined features.

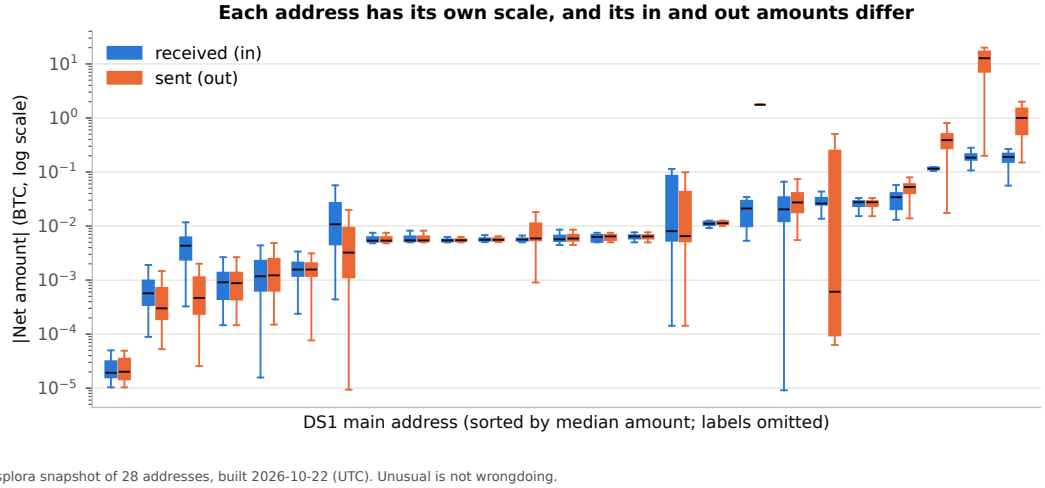


Figure 3: Received and sent amounts per DS1 main address (box: quartiles; line: median). Scales differ by four orders of magnitude, and many addresses send amounts unlike those they receive.

Finding 3: most transactions form a dense core. Two principal components explain 87.3% of the variance and 60.5% of transactions lie within a robust radius of 2 (Appendix F). Rank correlations are moderate (F1–F4 0.33, F2–F3 -0.57), so we keep all four features. A dense core with sparse outliers suggested distance- and density-based candidates.

Finding 4: labels are rare and drift over time. Elliptic's illicit share among labelled transactions is 11.6%

in the training steps but 6.5% in the test steps, and 77% of all rows are unlabelled. BitcoinHeist's positives are 1.4% of rows, and every feature separates them only weakly (largest rank-biserial effect 0.13). Accuracy would reward predicting the majority class (Chicco and Jurman 2020), so we split by time, report precision, recall, F1 and the area under the precision–recall curve (PR-AUC), and include class weights in tuning.

6 Machine Learning Model Selection

Criteria. A scoring method for the Explorer had to (1) work without labels, (2) judge each transaction against its own address's earlier history, (3) give a reason the user can check, (4) be stable, keeping a top-50 overlap (Jaccard) of at least 0.7 across refits, (5) flag no more than 20% of any address's transactions, and (6) run on a laptop CPU.

Candidates (Table 3). No detector wins on every kind of anomaly (Han et al. 2022), so each family covers a different idea of "unusual". The baseline flags a transaction when any feature lies far from the address's median. Clustering scores the distance to normal transactions (Ikotun et al. 2023; Bushra and Yi 2021). Regression predicts each amount and gap from earlier history and scores the miss; conformal calibration gives ranges with stated coverage (Angelopoulos and Bates 2023). Isolation Forest and the Local Outlier Factor (LOF) are general outlier detectors (Boukerche, Zheng and Alfandi 2021). Fusion combines the regression and Isolation Forest scores at a fixed flag budget. TxGPT, a small transformer trained from scratch on DS2, scores each transaction by its surprisal, the negative log-probability of what happened (Le and Zhang 2022). For labelled data we compared a linear model, a neighbour-based model and two tree ensembles, which remain strong on tabular data (Grinsztajn, Oyallon and Varoquaux 2022).

Table 3: Every model trained, by family. All were tuned on data that never produced a reported result.

Family	Models	Data	Output	Tuned
Statistics	Robust z-score baseline (A1)	DS1	Largest $\lvert z \rvert$ over F1–F4	Flag threshold
Clustering	K-Means; DBSCAN	DS1	Distance to normal clusters	k; eps, min. samples
Clustering	K-Means address profiles	DS1, DS2	Activity profile in words	k (silhouette)
Regression	Ridge; random forest; hist. gradient boosting	DS1	Miss on predicted amount and gap	Per-model grids
Regression	Quantile boosting + conformal	DS1	Distance outside a calibrated range	Quantiles, coverage
Outlier detection	Isolation Forest; LOF (novelty)	DS1	Isolation depth; local density	Trees, samples; neighbours
Combination	Score fusion	DS1	Weighted pair of scores	Weights at a flag budget
Classification	Logistic regression; KNN; random forest; hist. gradient boosting	DS3, DS4	Class probability	Validation split
Sequence model	TxGPT (trained from scratch)	DS2	Surprisal of each transaction	3 sizes $\times$ 3 seeds
Language model	Explorer Assistant, 152M (trained from scratch)	DS5–DS7	Explanation (guardrailed)	Adoption rule

Tuning and preliminary tests. Every search is in `tuning_*.csv`, run on plant seeds that never score reported results. K-Means' k came from the silhouette score and Davies–Bouldin index, DBSCAN's radius from the elbow of its k-distance graph, and each regressor was compared with a naive forecast (Table 6). Each method was then retrained in stages, and a stage was kept only if it passed a gate fixed in advance: higher recall with a bootstrap interval above zero, and no pattern falling by more than its seed spread. LOF, K-Means and DBSCAN passed; the other scoring and classifier stages failed and were dropped (`stages.csv`).

The selection rule, fixed in Assignment 1, picks the stable method with the highest mean planted recall, and within five points the easier one to explain. Fusion scored highest (0.682), but the baseline (0.655) lies

within the margin, and the difference is not significant (paired bootstrap, $p = 0.324$). The Explorer therefore uses the baseline, whose reasons map directly to the features.

Explanations. The *Explorer Assistant*, a language model, may phrase an explanation only if it passes a code guardrail: numbers from the facts, the limitation line, no banned word, correct comparison direction, and a verdict matching the flag. Otherwise a fixed template is shown, because language models can state numbers absent from their input (Ji et al. 2023).

7 Implementation

7.1 Technical implementation

The project is written in Python with pandas, scikit-learn and matplotlib (The pandas development team 2024; scikit-learn developers 2025; The Matplotlib Development Team 2025); PyTorch is used only for the two language models (Ansel et al. 2024). The `explorer` package has three layers (data and features, model execution, explanation), with one module per pipeline step, all run through one command-line tool; `config/settings.json` holds every threshold, window, grid and seed. Versions are pinned in a conda environment, and 273 tests check properties such as time safety and the guardrail. A full rebuild runs offline, and a script compares the rebuilt tables with the committed ones number by number (Pineau et al. 2021).

TxGPT was trained on GPUs in three sizes with three seeds each; the sizes tie, so the smallest (820,352 parameters) was kept. The Assistant follows a modern decoder design (Touvron et al. 2023) and was pretrained on 5.0 billion DS5 tokens, more than the compute-optimal budget of about 20 tokens per parameter (Hoffmann et al. 2022), then taught conversation (DS6) and explanations (DS7). It predicts on a CPU.

Engineering challenges. Filtering DS2's rows once per plant dominated the run time, so the table is split by address first (`explorer/inject.py`). Floating-point differences between machines changed the regression's fits, so `features.csv` is rounded to 12 significant digits. Long training survived interruptions because every trainer resumes from its own checkpoint.

The `Predictor` class is the interface Assignment 3 will call. Three commands expose it: `predict address` scores an address's whole timeline, `predict what-if` scores a hypothetical next transaction, and `predict explain` explains any transaction.

7.2 Implementation evaluation

Objectives. Table 4 compares the results with Assignment 1's targets. The baseline meets the recall target on spikes and dormant addresses but not on bursts counted per transaction (0.381), although it finds 0.776 of burst events, and four DS1 addresses break the sanity rule. Two objectives that need people outside the build remain open.

Scoring methods on DS1 (Table 5). Each method finds different patterns (Figure 4). Regression finds every planted burst event but few dormant addresses, LOF misses most spikes, and only fusion, the baseline and TxGPT catch most spikes, burst events and dormant addresses. TxGPT flags the fewest real transactions (1.5%) and so has the highest precision lower bound (0.421 against 0.125); a flag that is not a plant may still be worth a look, so real-data precision can only be bounded.

Regression and clustering (Table 6). The random forest cuts the naive forecast's error by 60% on gaps but only 10% on amounts. Conformal calibration lifts the amount range's coverage from 78.4% to 87.5% against a nominal 90%.

Agreement and stability over time. The methods flag different transactions: the baseline's overlap (Jaccard) with the other single methods is only 0.24–0.37, and 0.06 with TxGPT. Refitting at 4 walk-forward cut-offs keeps the baseline's mean recall at 0.650–0.658, so its result does not depend on one split; LOF varies more (0.331–0.410).

Table 4: Assignment 1's objectives for the selected method (the robust z baseline) and the results.

Objective (set in Assignment 1)	Result	Status
Snapshot of 20–30 main addresses (50+ tx, 90+ days) and 2–3 short ones	25 main and 3 short addresses	Met
Planted recall $\geq$ 70% for 2 of 3 patterns	Spike 0.744, dormant 0.840, burst 0.381	Met
Stability: top-50 Jaccard $\geq$ 0.7	1.000	Met
Sanity: no address over 20% flagged	DS1: 4 addresses over; DS10: 0	Partly met
Method agreement reported	Baseline–fusion Jaccard 0.512	Met
Blind reviewer check (Cohen's kappa)	No ratings recorded at submission; the blind sheet is in reports/reviewer	Open
Usability walkthrough with two participants	No session recorded at submission; moved to the Assignment 3 usability study	Open
Accuracy, precision, recall, F1 where labels exist	DS3 and DS4 (Table 7)	Met

Table 5: Scoring methods on DS1 (sorted by mean planted recall).

Method	Family	Spike	Burst	Dormant	Mean [95% CI]	Stab.	Flags
Score fusion	Combination	0.780	0.498	0.768	0.682 [0.573, 0.781]	0.736	9.4%
Robust z baseline (A1) $\star$	Statistics	0.744	0.381	0.840	0.655 [0.546, 0.753]	1.000	10.4%
Regression residual	Regression	0.864	0.695	0.320	0.626 [0.526, 0.722]	0.736	17.5%
TxGPT surprisal	Sequence model	0.736	0.170	0.736	0.547 [0.442, 0.638]	1.000	1.5%
DBSCAN distance	Clustering	0.544	0.109	0.776	0.476 [0.366, 0.571]	0.912	3.9%
K-Means distance	Clustering	0.624	0.050	0.736	0.470 [0.359, 0.567]	1.000	3.8%
Isolation Forest	Outlier detection	0.600	0.122	0.564	0.429 [0.321, 0.534]	0.876	3.6%
Local Outlier Factor	Outlier detection	0.116	0.066	0.812	0.331 [0.262, 0.392]	0.846	4.5%
Regression range (conformal)	Regression	0.712	0.174	0.044	0.310 [0.246, 0.373]	0.514	4.8%

Recall on planted transactions over five reporting seeds; intervals are 95% address-level bootstrap. Burst is per transaction; mean recall averages spike, burst and dormant. Stability is the top-50 Jaccard over refits. Flag rate is the share of real evaluation-period transactions flagged. $\star$ the selected method.

Table 6: Regression and clustering metrics on DS1.

Target	Model	MAE	RMSE	R^2	Skill
Amount	Naive forecast	0.186	0.416	0.730	0.000
Amount	Ridge	0.181	0.373	0.783	0.027
Amount	Random forest	0.168	0.368	0.788	0.099
Amount	Hist. gradient boosting	0.189	0.389	0.764	-0.015
Gap	Naive forecast	0.476	0.665	0.039	0.000
Gap	Ridge	0.279	0.410	0.635	0.414
Gap	Random forest	0.189	0.301	0.803	0.603
Gap	Hist. gradient boosting	0.192	0.298	0.807	0.597

Regression on the evaluation period, log10 scale; skill is $1 - $ MAE / the naive forecast's MAE. Clustering, first search before the stages: K-Means $k = 2$, silhouette 0.657, Davies–Bouldin 0.834; DBSCAN radius 1.3$\times$ the k-distance elbow, silhouette 0.786 without noise points, 2.0% noise.

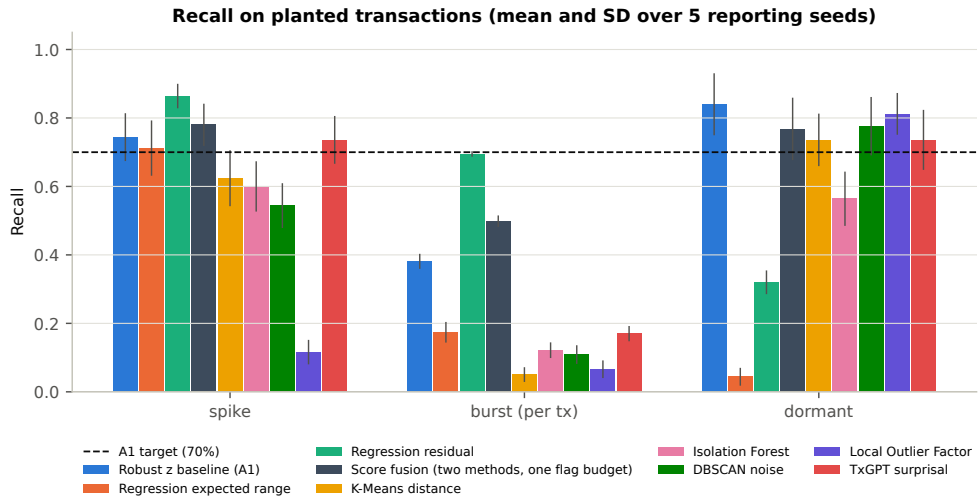


Figure 4: Recall on planted transactions by pattern (mean and standard deviation over five reporting seeds). No method is best at every pattern; the dashed line is Assignment 1's 70% target.

Unseen addresses: DS2 and DS10. The frozen methods were tested on 2,933 DS2 histories and once on DS10 (Figure 5). Two outcomes were unexpected. First, methods tuned on DS1 fall the most: fusion drops from 0.682 to 0.367 on DS10, below the baseline's 0.431. Second, TxGPT, which learned from other addresses, ranks first on DS10 (0.598, interval [0.534, 0.657]). Over 100 plant seeds the baseline ranks first among the pooled methods on DS10 in 96% of seeds. Bursts are hard to find on DS10, whose addresses are busier and less regular (median gap 13.1 hours against 23.1 on DS1).

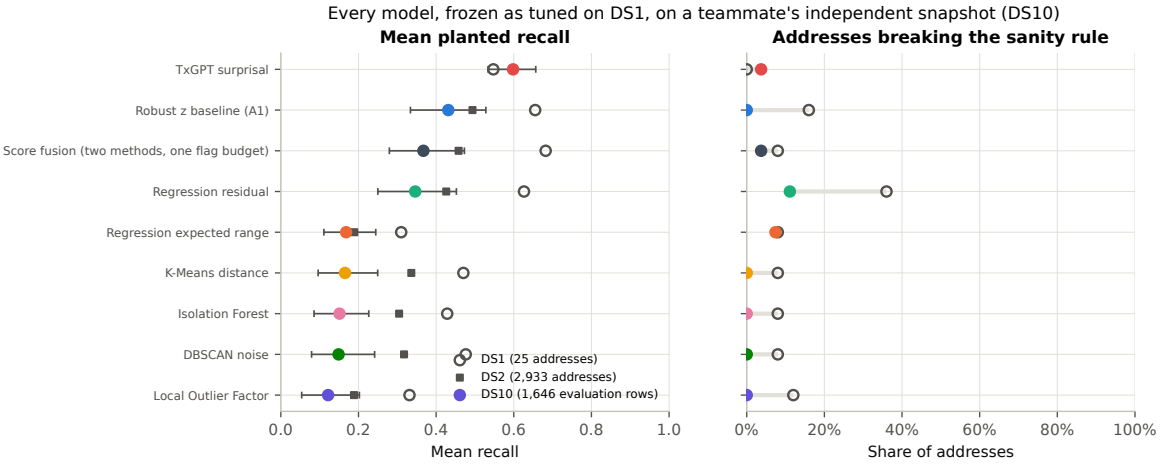


Figure 5: Every method, frozen as tuned on DS1, on DS2 and the held-out DS10 (left: mean planted recall with 95% bootstrap intervals on DS10; right: share of addresses breaking the sanity rule).

Classifiers on labelled data (Table 7). On Elliptic the random forest has the best F1 (0.815, interval [0.796, 0.835] over ten training seeds), but every model collapses from time step 43 (Figure 13, Appendix F), when a dark market closed and illicit behaviour changed (Pareja et al. 2020). On BitcoinHeist's temporal split no model's F1 exceeds 0.07, and families held out of training are found at recalls of 0.3%–3.3% by the tree models, consistent with Finding 4.

Unusual is not a label. Scored against the labelled classes, the unsupervised methods do worse than chance (PR-AUC below the class share), while the random forest reaches 12.2× chance on Elliptic (Appendix F). Being unusual is not the same as belonging to a labelled class.

Language models. The Assistant passes the guardrail on 100.0% of DS7 test prompts and 100.0% of DS10 prompts, and is fully correct (reasons, order, numbers) on 96.9% and 93.1%; pooled over twelve test

Table 7: Classifiers on the labelled datasets.

Data and split	Model	Accuracy	Precision	Recall	F1	PR-AUC
Elliptic, all features	Hist. gradient boosting	0.971	0.808	0.734	0.769	0.803
Elliptic, all features	k-nearest neighbours	0.957	0.684	0.620	0.650	0.659
Elliptic, all features	Logistic regression	0.949	0.619	0.578	0.598	0.412
Elliptic, all features	Random forest	0.979	0.928	0.727	0.815	0.792
Elliptic, local features	Hist. gradient boosting	0.965	0.738	0.721	0.730	0.784
Elliptic, local features	k-nearest neighbours	0.969	0.792	0.703	0.745	0.741
Elliptic, local features	Logistic regression	0.969	0.839	0.640	0.726	0.619
Elliptic, local features	Random forest	0.970	0.801	0.721	0.759	0.781
BitcoinHeist, temporal	Hist. gradient boosting	0.705	0.012	0.722	0.024	0.042
BitcoinHeist, temporal	k-nearest neighbours	0.789	0.011	0.452	0.021	0.025
BitcoinHeist, temporal	Logistic regression	0.170	0.006	0.976	0.012	0.005
BitcoinHeist, temporal	Random forest	0.942	0.038	0.436	0.070	0.052
BitcoinHeist, random	Hist. gradient boosting	0.975	0.199	0.252	0.222	0.151
BitcoinHeist, random	k-nearest neighbours	0.957	0.099	0.254	0.143	0.071
BitcoinHeist, random	Logistic regression	0.510	0.020	0.691	0.039	0.017
BitcoinHeist, random	Random forest	0.979	0.250	0.239	0.244	0.177

Test splits: Elliptic time steps 35–49; BitcoinHeist years after the training years (temporal) or a random 20%. Metrics are for the datasets' own positive classes (illicit; ransomware), at the threshold chosen on validation data.

sets, 94.8% (95% interval [94.1%, 95.4%]). Its weakest area is open glossary questions, where it states 48% of the key facts.

8 Conclusion

Our pipeline ranks an address's transactions by how unusual each is for that address. Our answers:

RQ1 The baseline is the Explorer's method: within the margin of the best method on DS1, stable, explainable, and the most consistent on unseen addresses.

RQ2 Classifiers do well on Elliptic until behaviour changes, and stay weak on BitcoinHeist.

RQ3 Unusual activity and the labelled illicit class barely overlap: a flag is a reason to look, not a verdict.

RQ4 TxGPT, trained only on other addresses, ranks first on the held-out DS10; the Assistant explains flags faithfully under a guardrail, though it is weak on open questions.

Impact and limitations. For its users, the Explorer reduces an unranked history to the 10.4% of transactions worth checking first, each with a reason, though no user study has tested this yet. Plants are our guess at what unusual looks like; DS1 has only 28 addresses, so intervals are wide; and a language model can still give a wrong reason the guardrail misses, so the template remains the default.

The main insight is that methods tuned on one sample of addresses lose much of their advantage on another, so an independent held-out test was essential. Future work for Assignment 3 includes a web dashboard on the prediction interface, live block data, TxGPT's surprisal beside the baseline, explanations refined from user feedback, and a usability study with the intended users.

Bibliography

[1] Akcora, C.G., Li, Y., Gel, Y.R. and Kantarcioglu, M. (2020) 'BitcoinHeist: topological data analysis for ransomware prediction on the Bitcoin blockchain', in *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI-20)*, pp. 4439–4445. doi: 10.24963/ijcai.2020/612.

[2] Angelopoulos, A.N. and Bates, S. (2023) 'Conformal prediction: a gentle introduction', *Foundations and Trends in Machine Learning*, 16(4), pp. 494–591. doi: 10.1561/2200000101.

[3] Ansel, J. et al. (2024) 'PyTorch 2: faster machine learning through dynamic Python bytecode transformation and graph compilation', in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*, pp. 929–947. doi: 10.1145/3620665.3640366.

[4] Ben Allal, L. et al. (2025) 'SmolLM2: when smol goes big — data-centric training of a small language model', *arXiv preprint*, arXiv:2502.02737.

[5] Blázquez-García, A., Conde, A., Mori, U. and Lozano, J.A. (2022) 'A review on outlier/anomaly detection in time series data', *ACM Computing Surveys*, 54(3), pp. 1–33. doi: 10.1145/3444690.

[6] Blockstream (n.d.) *Esplora HTTP API*. Available at: https://github.com/Blockstream/esplora/blob/master/API.md (Accessed: 22 October 2026).

[7] Boukerche, A., Zheng, L. and Alfandi, O. (2021) 'Outlier detection: methods, models, and classification', *ACM Computing Surveys*, 53(3), pp. 1–37. doi: 10.1145/3381028.

[8] Bushra, A.A. and Yi, G. (2021) 'Comparative analysis review of pioneering DBSCAN and successive density-based clustering algorithms', *IEEE Access*, 9, pp. 87918–87935. doi: 10.1109/ACCESS.2021.3089036.

[9] Chicco, D. and Jurman, G. (2020) 'The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation', *BMC Genomics*, 21, 6. doi: 10.1186/s12864-019-6413-7.

[10] Elliptic (2019) *Elliptic Data Set* [Dataset]. Kaggle. Available at: https://www.kaggle.com/datasets/ellipticco/elliptic-data-set (Accessed: 22 October 2026).

[11] Elmougy, Y. and Liu, L. (2023) 'Demystifying fraudulent transactions and illicit nodes in the Bitcoin network for financial forensics', in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*, pp. 3979–3990. doi: 10.1145/3580305.3599803.

[12] Grinsztajn, L., Oyallon, E. and Varoquaux, G. (2022) 'Why do tree-based models still outperform deep learning on typical tabular data?', *Advances in Neural Information Processing Systems*, 35, Datasets and Benchmarks Track. arXiv:2207.08815.

[13] Han, S., Hu, X., Huang, H., Jiang, M. and Zhao, Y. (2022) 'ADBench: anomaly detection benchmark', *Advances in Neural Information Processing Systems*, 35, Datasets and Benchmarks Track. arXiv:2206.09426.

[14] Hilal, W., Gadsden, S.A. and Yawney, J. (2022) 'Financial fraud: a review of anomaly detection techniques and recent advances', *Expert Systems with Applications*, 193, 116429. doi: 10.1016/j.eswa.2021.116429.

[15] Hoffmann, J. et al. (2022) 'An empirical analysis of compute-optimal large language model training', *Advances in Neural Information Processing Systems*, 35. arXiv:2203.15556.

[16] Ikotun, A.M., Ezugwu, A.E., Abualigah, L., Abuhaija, B. and Heming, J. (2023) 'K-means clustering algorithms: a comprehensive review, variants analysis, and advances in the era of big data', *Information Sciences*, 622, pp. 178–210. doi: 10.1016/j.ins.2022.11.139.

[17] Ji, Z. et al. (2023) 'Survey of hallucination in natural language generation', *ACM Computing Surveys*, 55(12), pp. 1–38. doi: 10.1145/3571730.

[18] Kapoor, S. and Narayanan, A. (2023) 'Leakage and the reproducibility crisis in machine-learning-based science', *Patterns*, 4(9), 100804. doi: 10.1016/j.patter.2023.100804.

[19] Lai, K.-H., Zha, D., Xu, J., Zhao, Y., Wang, G. and Hu, X. (2021) 'Revisiting time series outlier detection: definitions and benchmarks', *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 1 (NeurIPS 2021). Available at: https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/ec5decca5ed3d6b8079e2e7e7bacc9f2-Abstract-round1.html (Accessed: 29 October 2026).

[20] Le, V.-H. and Zhang, H. (2022) 'Log-based anomaly detection with deep learning: how far are we?', in *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*, pp. 1356–1367. doi: 10.1145/3510003.3510155.

[21] mempool.space (n.d.) *REST API documentation*. Available at: https://mempool.space/docs/api/rest (Accessed: 22 October 2026).

[22] Pareja, A. et al. (2020) 'EvolveGCN: evolving graph convolutional networks for dynamic graphs', *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(04), pp. 5363–5370. doi: 10.1609/aaai.v34i04.5984.

[23] Penedo, G. et al. (2024) 'The FineWeb datasets: decanting the web for the finest text data at scale', *Advances in Neural Information Processing Systems*, 37, pp. 30811–30849. doi: 10.52202/079017-0970.

[24] Pineau, J. et al. (2021) 'Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)', *Journal of Machine Learning Research*, 22(164), pp. 1–20.

[25] scikit-learn developers (2025) *scikit-learn* (Version 1.6.1) [Software]. Zenodo. doi: 10.5281/zenodo.14627164.

[26] The Matplotlib Development Team (2025) *Matplotlib: visualization with Python* (Version 3.10.1) [Software]. Zenodo. doi: 10.5281/zenodo.14940554.

[27] The pandas development team (2024) *pandas-dev/pandas: Pandas* (Version 2.2.3) [Software]. Zenodo. doi: 10.5281/zenodo.13819579.

[28] Touvron, H. et al. (2023) 'LLaMA: open and efficient foundation language models', *arXiv preprint*, arXiv:2302.13971.

[29] UCI Machine Learning Repository (2020) *Bitcoin Heist Ransomware Address* [Dataset]. doi: 10.24432/C5BG8V. Available at: https://archive.ics.uci.edu/dataset/526/bitcoinheistransomwareaddressdataset (Accessed: 20 October 2026).

A Requirements checklist

The brief's requirements for this report and two objectives from Assignment 1, with where each is met.

Requirement	Where it is met	Status
Project title, group name and ID, students' names and IDs, tutor	Cover	✓
Motivation and intended users	Section 1	✓
Challenge, limits of existing solutions, why machine learning fits	Section 2	✓
Sources, methods, filtering criteria and challenges of collection	Section 3, Table 1	✓
Cleaning, missing values, normalisation, feature engineering, joining	Section 4, Table 2	✓
Analysis with statistics and visualisation	Section 5, `notebooks/01_eda.ipynb`	✓
Selection criteria, alternatives and preliminary tests	Section 6, Table 3	✓
At least two of classification, clustering and regression	All three, Table 3	✓
Technical implementation: architecture, libraries, code, challenges	Section 7.1, Appendices B and D	✓
Evaluation against objectives, with unexpected outcomes	Section 7.2, Tables 4–7	✓
Accuracy, precision, recall and F1	Table 7, where labels exist	✓
Conclusion: outcomes, impact on users, limitations, future work	Section 8	✓
Harvard bibliography	Bibliography	✓
Additional datasets	DS2–DS7 and DS10, Table 1	✓
Additional machine-learning implementations	Table 3	✓
Blind reviewer check (Assignment 1)	Table 4	Open
Usability walkthrough (Assignment 1)	Table 4	Open

B System architecture and workflow

Figure 6 shows how the modules fit together. Every step reads its thresholds from one settings file, and this report's results are read from the result tables by a script.

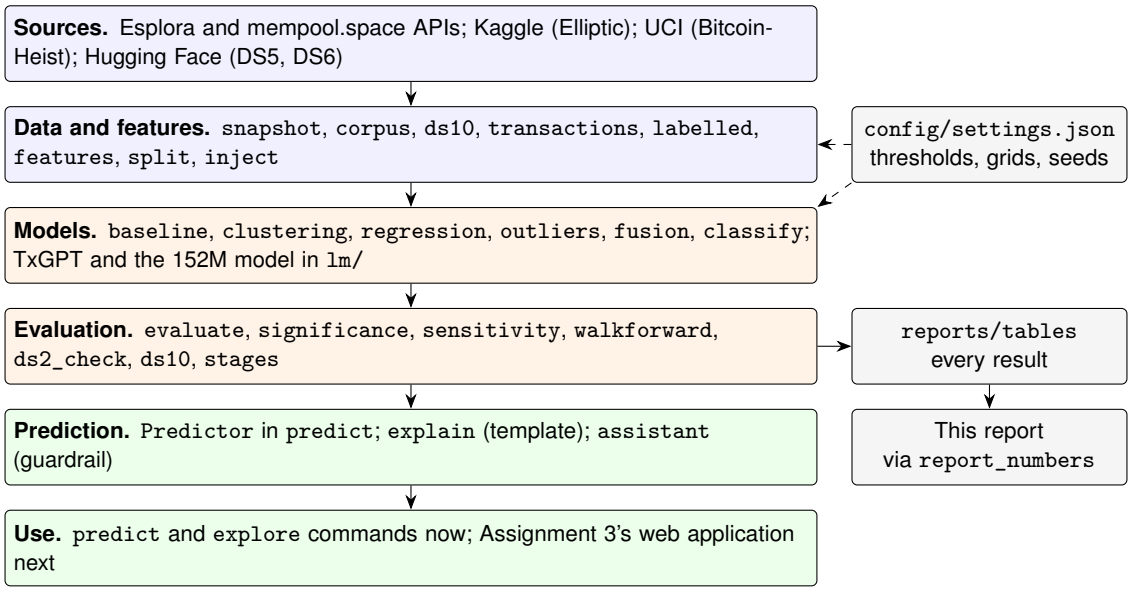


Figure 6: System architecture. Names in code type are modules of `explorer/` unless marked.

Figure 7 shows the path a user takes through the interactive Explorer. It serves these user stories:

- As a student researcher, I want to see which of an address's transactions stand out, so that I know where to start reading.
- As a compliance analyst, I want the reason beside each flag, so that I can check it against the address's history before acting.
- As a developer on Assignment 3, I want one class that scores and explains, so that the web application needs no model code of its own.

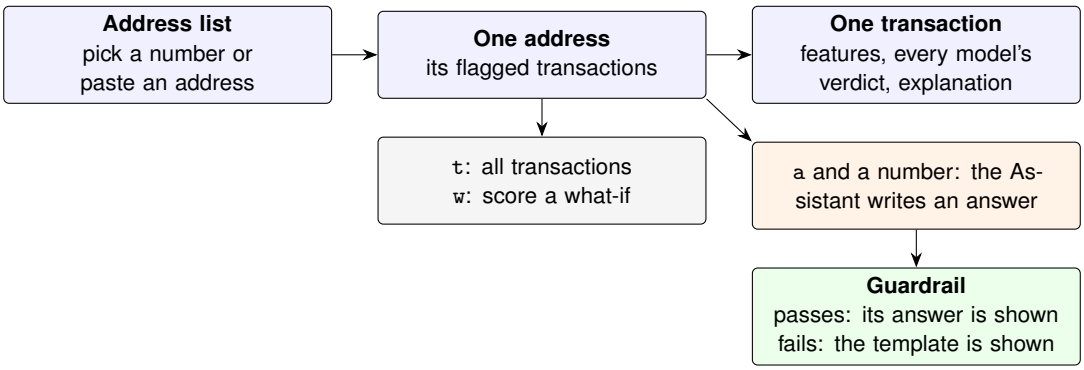


Figure 7: The interactive Explorer's workflow (`python -m explorer.cli explore`).

C Feature walkthroughs

Browse an address and ask why

Purpose. Find the transactions worth a look in one address's history, and see the reason for each.

Step 1. Run `python -m explorer.cli explore`. The list shows each snapshot address with its counts.

```
Bitcoin Address Activity Explorer
Snapshot of 28 addresses from Blockstream Esplora, taken 2026-10-22.
Flags are decided by Robust z baseline (A1): any feature with |z| at least 5.
Unusual activity requires investigation; it does not indicate wrongdoing.

Addresses (page 1 of 3)
  #  Address                                            Tx  Scored  Flagged  First       Last
  1  36V7afbhZnvpMpBJmZf7XJKDeL2vtHDB6L                700     683       26  2025-10-12  2026-10-22
  2  3HTmUj5wFDQZ2TF8SAcngV7EQ88rptx35D                 677     667       97  2020-12-25  2026-10-19
  3  35ZCxf5uyfc2VN194t8WRzz6RCEN6tQfmY                 558     548       84  2024-10-15  2026-10-22
  4  bc1q7evspfauqve6ll84juuj0anlzrxxpdeq7raqn3         507     455       36  2026-06-07  2026-10-22
  5  bc1qxsq47e8qacpx04e0tdxr6m4kmdsapt3weul9jh         455     440       42  2025-07-06  2026-10-22
  6  1GABHZfEcFEZEYWQNsnq9HFEwEzV6CT4Ku                 426     412       14  2024-10-08  2026-10-22
  7  1Bbcsfxj2JRUD5crcdZVHBcViT61vrJ3dz                 377     367        3  2024-05-16  2026-10-19
  8  3Aw7V5ibs39eL3otstFN4wrwa6VAN9BKhz                 349     339       49  2022-01-28  2026-10-19
  9  bc1qu6pzmup040ufksvs9x06ther9zag9tj9l68tc5         305     290       19  2025-11-18  2026-10-22
 10  bc1qsxjuxwlyyc4shfg3r6c5djjf8ehzqftt7zzcl6         289     278       13  2026-01-02  2026-10-21
```

Step 2. Type an address's number, here 7. Its flagged transactions appear, each with its main reason.

```
Address 1Bbcsfxj2JRUD5crcdZVHBcViT61vrJ3dz
377 transactions from 2024-05-16 to 2026-10-19; 367 scored, 3 flagged on 3 days.

Flagged transactions (3)
    #  Time (UTC)        Dir              Amount   Score  Flag  Main reason
   11  2024-07-11 03:32  in        0.0011126 BTC    5.61  FLAG  Lower ratio to the usual amount than usual (z -5.6)
  193  2026-01-09 00:34  out       0.00239392 BTC   5.19  FLAG  Higher ratio to the usual amount than usual (z 5.2)
  365  2026-09-28 06:51  in        0.00056497 BTC   5.11  FLAG  Longer gap than usual (z 5.1)
```

Step 3. Type a transaction's number, here 11. The screen compares its features with the address's own history, lists every saved model's verdict, and gives the explanation.

```
Transaction #11: 2024-07-11 03:32 UTC, in, 0.0011126 BTC
  https://blockstream.info/tx/c91d9fbbd396070099f3147822cd6e1eb97b1de376ad353f42007149ddb73140

Compared with this address's own history (the flag line is |z| 5):
  Feature         This transaction        Usual                      z
  Size            0.0011126 BTC           0.00291708 BTC          -2.9  <- stands out
  Burst           1 transaction           1.0 transactions         0.0
  Gap             48 hours                6 days                  -1.8
  Relative size   0.4x                    1.4x                    -5.6  <- over the flag line

Every saved model's verdict (1 of 9 that could score it flag it; the Explorer shows Robust z baseline (A1)'s):
  FLAG  Robust z baseline (A1)                        5.61  statistics: |z| against the address's own median and MAD (A1's rule; nothing trained) (decides the flag)
  -     Isolation Forest                              0.52  outlier detection: random isolation trees
  -     Local Outlier Factor                          1.47  outlier detection: local density against the nearest neighbours
  -     K-Means distance                              6.69  clustering: distance to the nearest K-Means centre
  -     DBSCAN noise                                  1.70  clustering: distance to DBSCAN's nearest dense core
  -     Regression residual                           2.57  regression: the miss between the predicted and actual size and gap
  -     Regression expected range                    -0.16  regression: distance outside the conformal expected range
  -     Score fusion (two methods, one flag budget)   0.99  combination: regression and Isolation Forest scores together
  -     TxGPT surprisal                               5.57  language model: TxGPT's surprisal (trained from scratch on DS2)

Explanation, written by the template (fixed sentences filled in from the numbers above; no model):
  Flagged by Robust z baseline (A1).
  What stands out compared with this address's own history:
  - Lower ratio to the usual amount than usual: 0.4x the usual amount for this direction, against a usual 1.4x.
  - Smaller amount than usual: 0.0011126 BTC moved, against a usual 0.00291708 BTC.
  Unusual activity requires investigation; it does not indicate wrongdoing.
```

Test a hypothetical transaction

Purpose. Ask how a transaction that has not happened would compare with an address's real history, the user-input prediction Assignment 3 will offer.

Step 1. Run `predict what-if` with an address, an amount, a direction and a time, or type `w` in the Explorer.

Step 2. Read whether it is flagged and which features stand out (Appendix E). TxGPT sits this one out, because its scores are made in advance.

Ask the Assistant to explain

Purpose. Have the language model phrase an explanation, without trusting it blindly.

Step 1. Add `--assistant` to `predict explain`, or type `a` and a number in the Explorer.

Step 2. Read the `source` field, which says who wrote the text. An answer that fails the guardrail is never shown: the template appears instead, and `failedChecks` lists what went wrong.

Step 3. Without the model's weights on the machine, `skippedWriters` says so and the template answers.

D Key code

Time safety. Transactions in the same block share a time, so a transaction's history ends before the first transaction at its own time. This is the rule behind every feature (Section 4).

`explorer/features.py`, lines 190–193

```
    # Earlier transactions of i are indices [0, earlier_end[i]): same-time transactions are excluded.
    earlier_end: list[int] = []
    for i, t in enumerate(txs):
        earlier_end.append(earlier_end[i - 1] if i > 0 and txs[i - 1].time == t.time else i)
```

A hypothetical transaction. The same rule holds for a what-if: only real transactions strictly before its time form its history.

`explorer/predict.py`, lines 237–244

```
    def score_what_if(self, address: str, amount_btc: float, direction: str, time_utc: str) -> dict[str, Any]:
        """Score a hypothetical transaction against the address's real, strictly earlier history."""
        if direction not in ("in", "out"):
            raise ValueError("direction must be 'in' or 'out'")
        self.history_of(address)
        real = self.transactions[self.transactions["address"] == address]
        t = int(datetime.fromisoformat(time_utc.replace("Z", "+00:00")).timestamp())
        earlier = real[real["block_time_utc"] < t]
```

The guardrail. An answer must pass every check before the user sees it (Section 6).

`explorer/assistant.py`, lines 264–272

```python
def check(answer: str, messages: list[dict[str, str]], facts: dict[str, Any]) -> dict[str, Any]:
    """The guardrail's five checks on one answer, and whether it passes all of them."""
    prompt = "\n".join(m["content"] for m in messages)
    result = {"numbers_ok": numbers_faithful(answer, prompt),
              "limitation_ok": has_limitation(answer) or not facts.get("flagged", False),
              "wording_ok": not banned_words(answer), "direction_ok": not wrong_directions(answer, facts),
              "verdict_ok": verdict_ok(answer, facts), "nonempty": bool(answer.strip())}
    result["passed"] = all(result.values())
    return result
```

E Example prediction

The output below is what Assignment 3's back end receives. It was produced by `python -m explorer.cli predict what-if` on a hypothetical transaction of one DS1 address: sending 0.05 BTC, about a hundred times the address's usual amount. The size features (F1, F4) pass the flag line; the timing features (F2, F3) are within the usual range, so they are not mentioned.

```
input:    0.05 BTC out at 2026-10-20T12:00Z, address 1BbcsfxjJRUD5crcdZVHBcViT61vrJ3dz
scored: true   flagged: true   score (largest |z|): 5.98
feature                     value        usual       z  role
F1 Size                      0.05  0.000524948    5.88  flag
F2 Burst                        3            4   -0.45  within-usual
F3 Gap                    21.8264      10.0535    0.30  within-usual
F4 Relative size          95.2472     0.909938    5.98  flag
```

Explanation shown to the user (template writer):

```
Flagged by Robust z baseline (A1).
What stands out compared with this address's own history:
- Higher ratio to the usual amount than usual: 95.2x the usual amount for this direction, against a
  usual 0.9x.
- Larger amount than usual: 0.05 BTC moved, against a usual 0.00052495 BTC.
Unusual activity requires investigation; it does not indicate wrongdoing.
```

F Further results

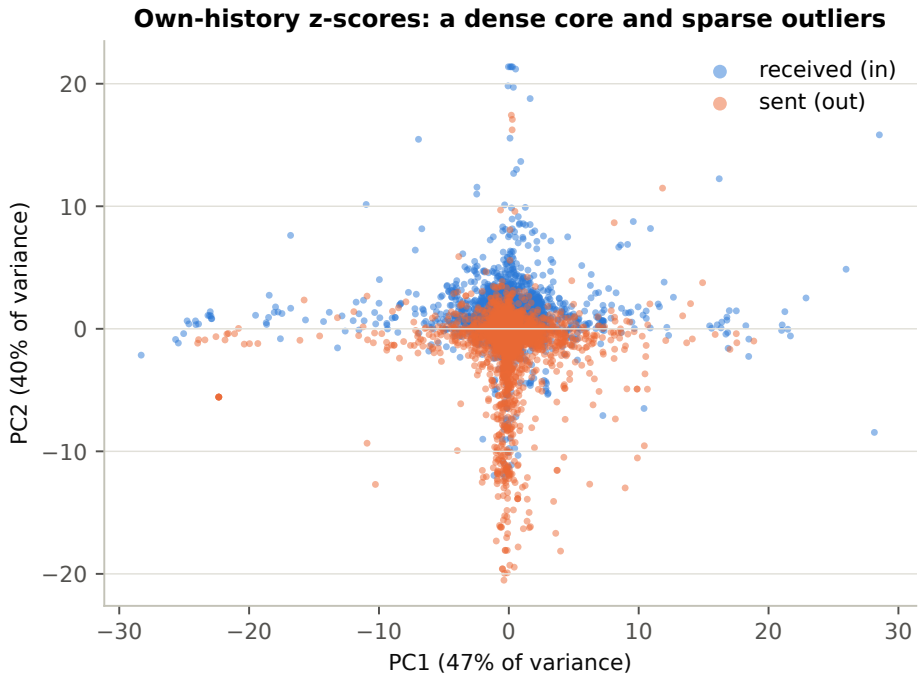


Figure 8: DS1's transactions on the first two principal components, showing a dense core with sparse outliers (Finding 3).

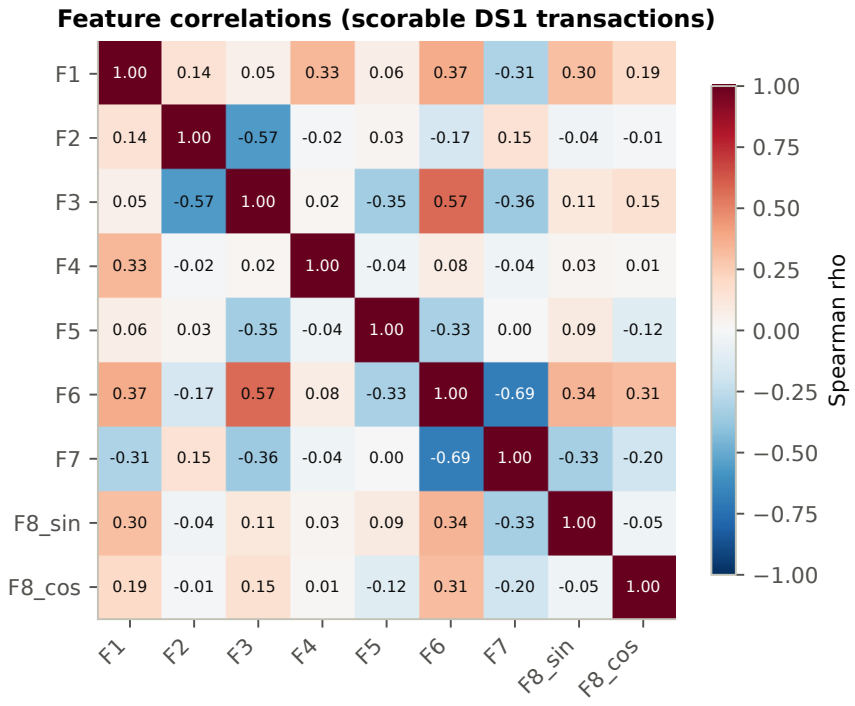


Figure 9: Rank correlations between DS1's features (Finding 3).

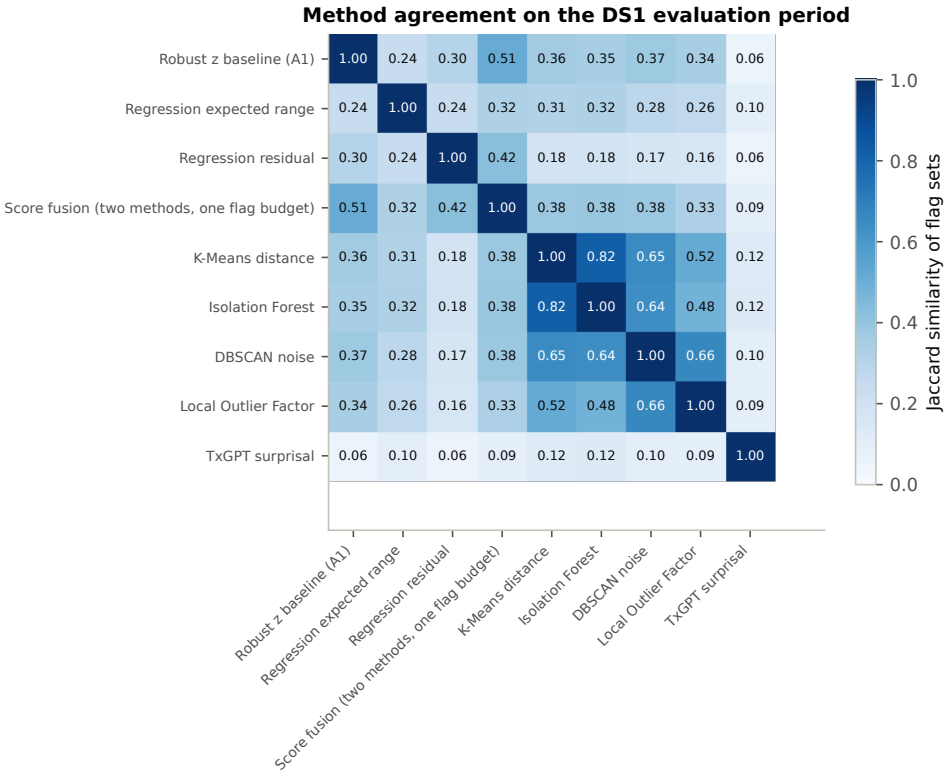


Figure 10: Pairwise Jaccard overlap between the methods' flagged transactions (`eval_agreement.csv`).

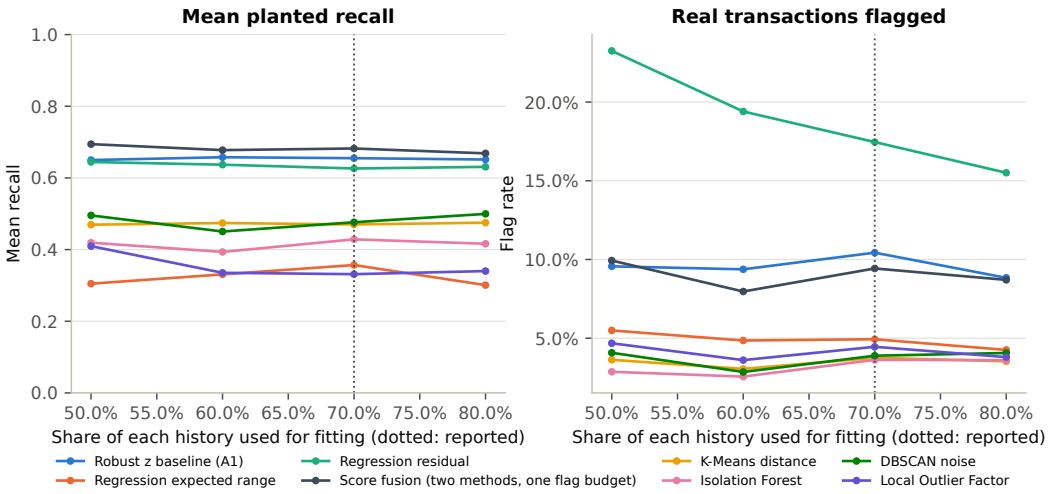


Figure 11: Every method refitted at each walk-forward cut-off (`eval_walkforward.csv`).

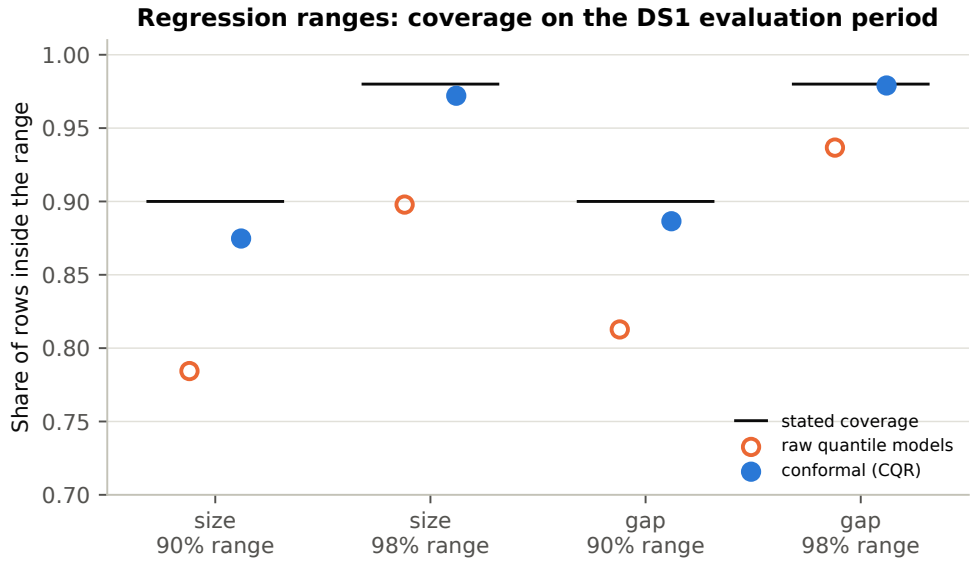


Figure 12: Coverage of the regression ranges before and after conformal calibration (`regression_results.csv`).

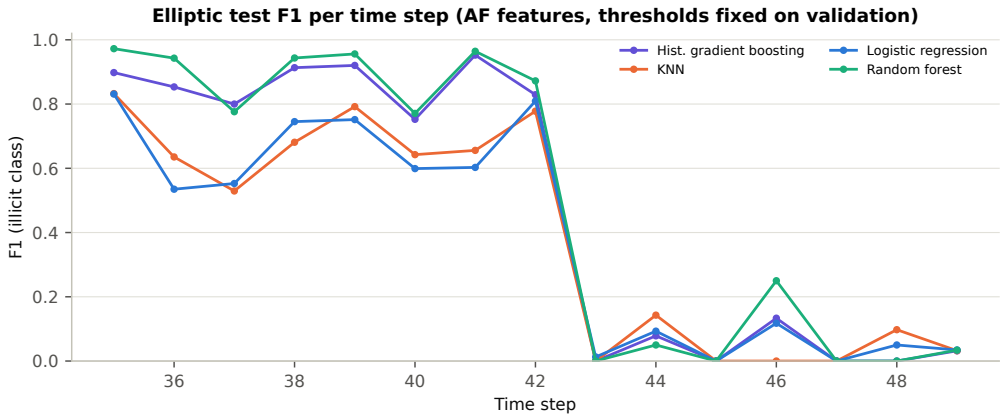


Figure 13: Elliptic test F1 per time step. Every model fails from step 43, after the shutdown of a dark market changed the illicit class's behaviour.

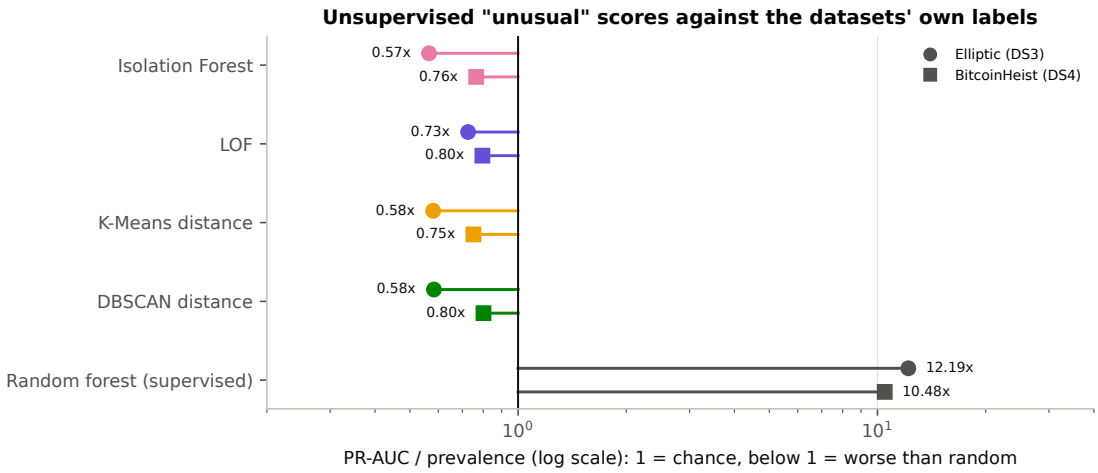


Figure 14: Unsupervised scores against the labelled datasets' own classes. Every unsupervised method is below chance (1×), while the supervised random forest is far above it: being unusual is not the same as belonging to a labelled class (`label_check.csv`).

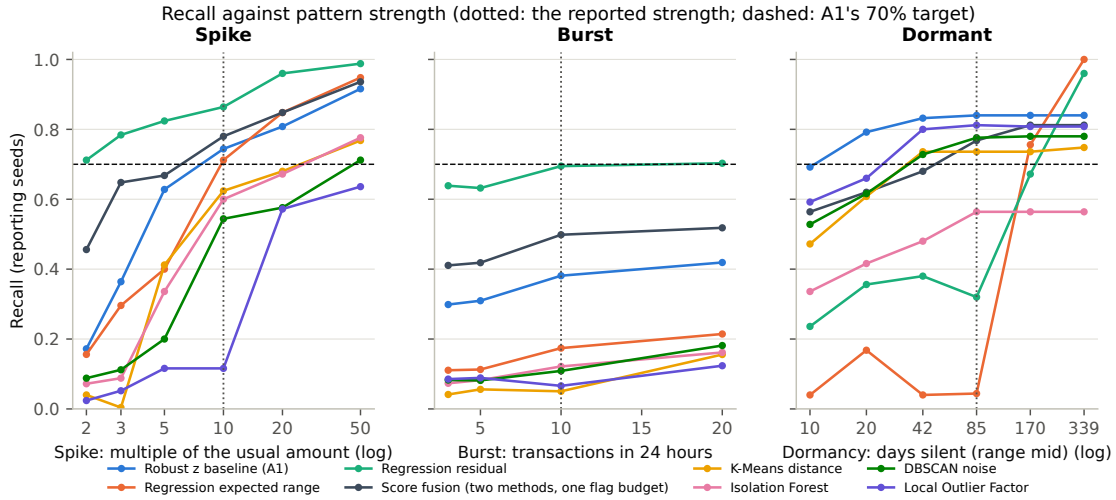


Figure 15: Recall against the strength of each planted pattern (`eval_sensitivity.csv`). Weaker plants are harder for every method; regression keeps finding small spikes that the baseline misses.

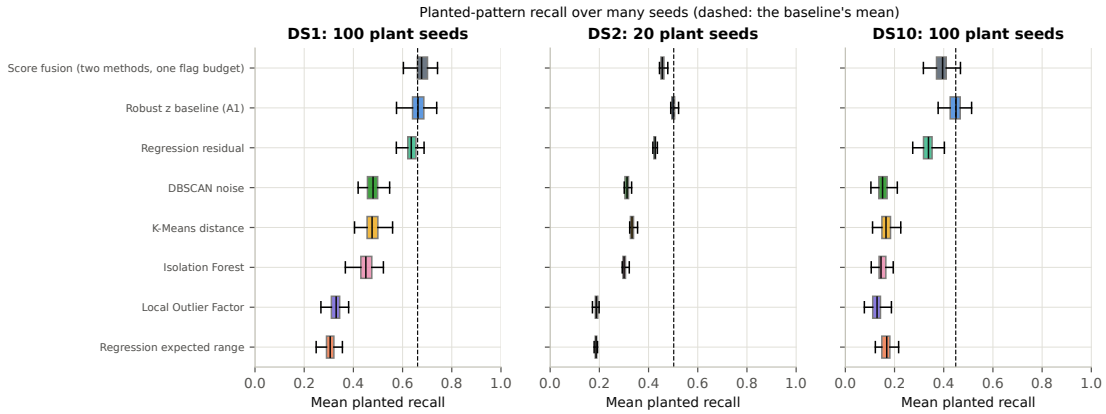


Figure 16: Mean planted recall over 100 plant seeds on DS1 and DS10 and 20 on DS2 (`stats_methods_summary.csv`).

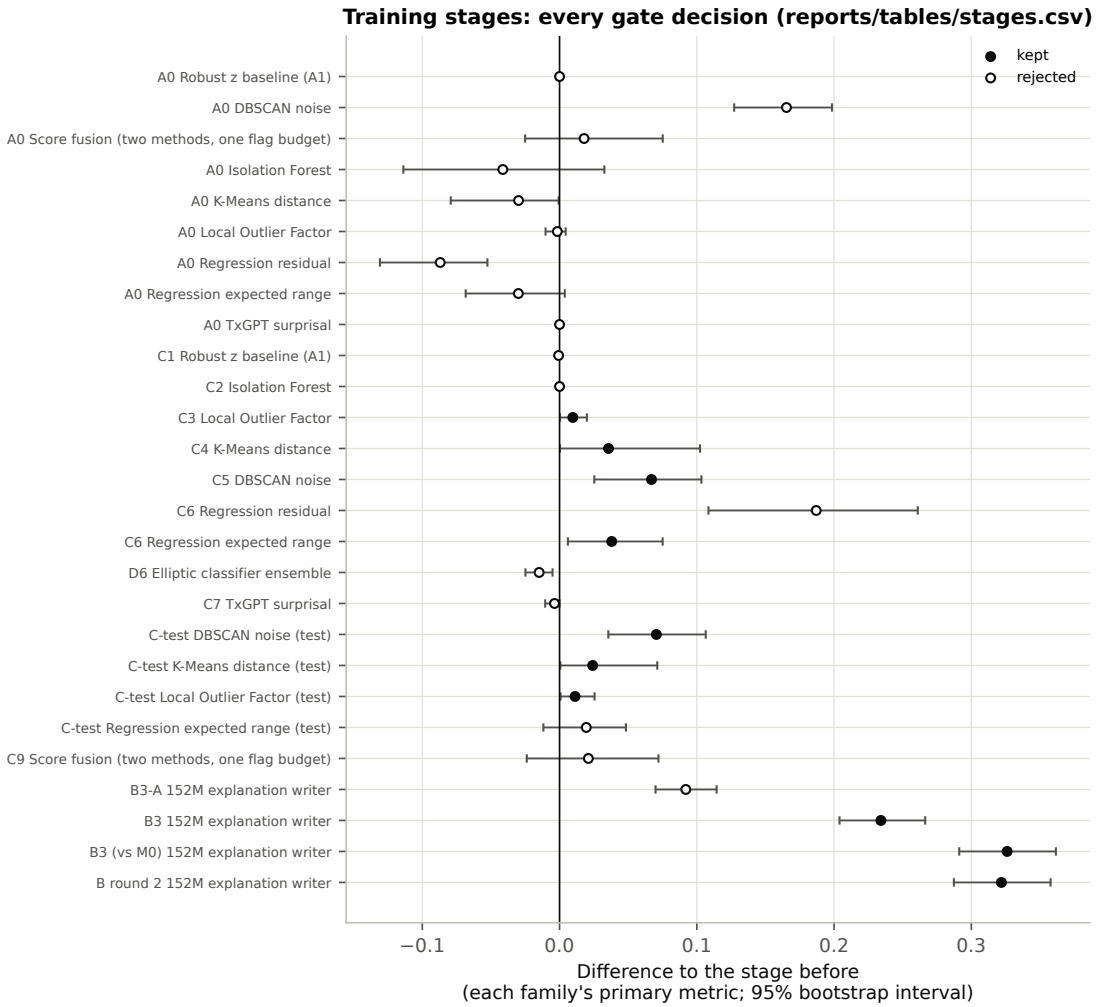


Figure 17: Every training stage's gate decision with its bootstrap interval (`stages.csv`).

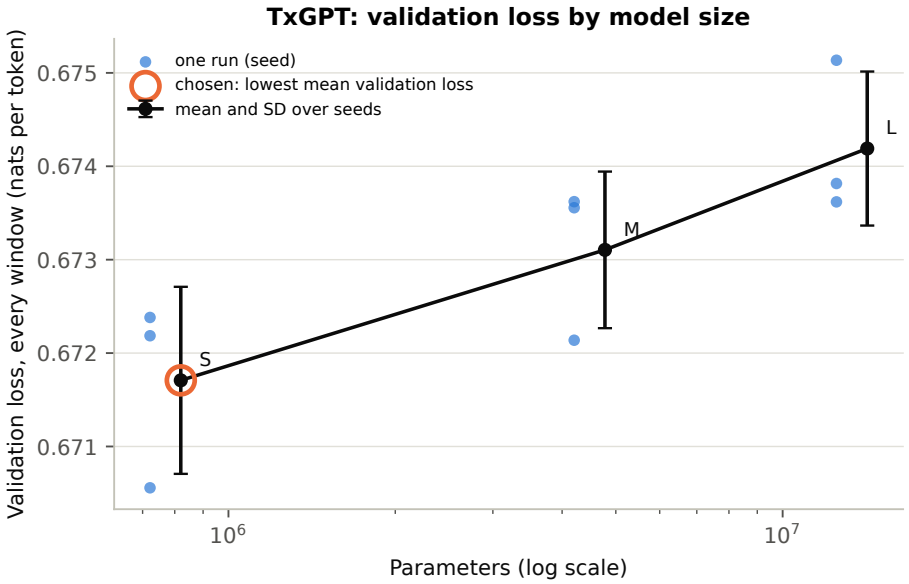


Figure 18: TxGPT's validation loss for three sizes and three seeds each (`txgpt_sizes.csv`). The sizes tie, so the smallest was kept.

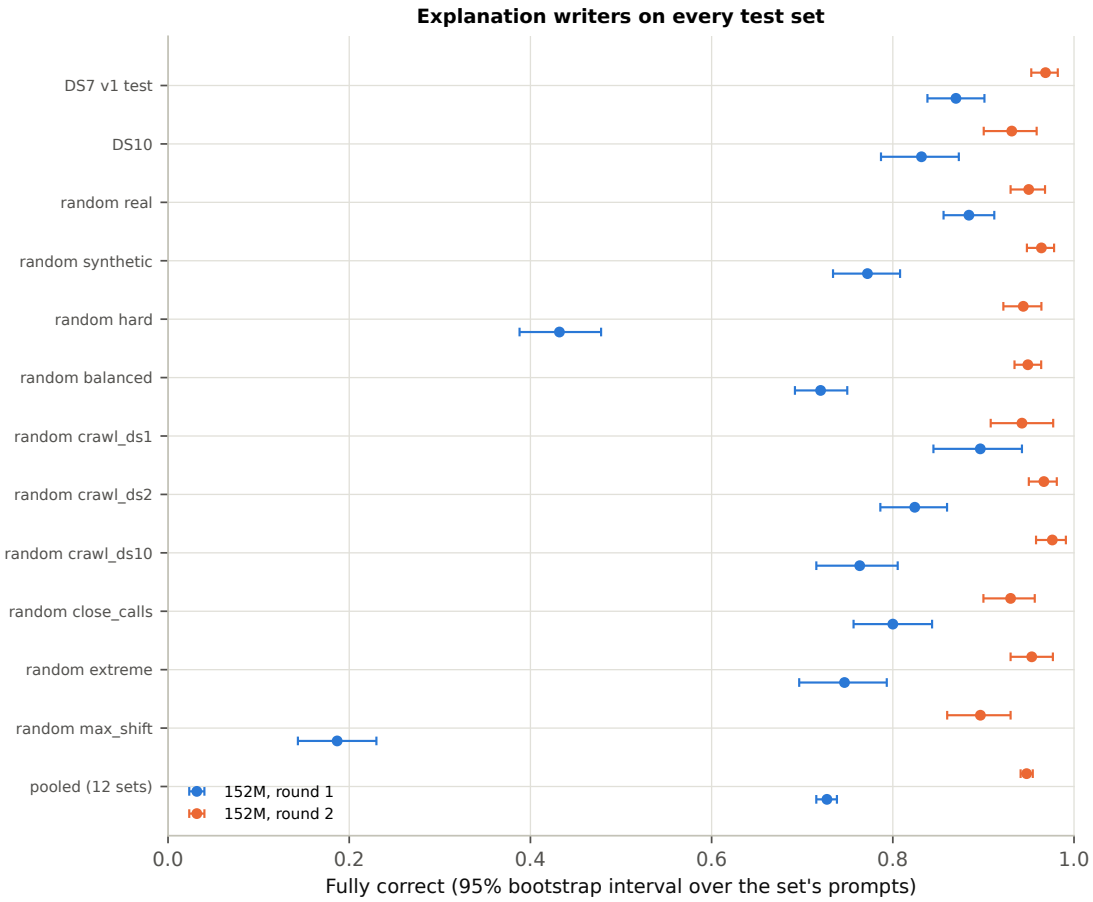


Figure 19: The Assistant's fully correct rate on each test set, before (round 1) and after (round 2) its second training round, with 95% bootstrap intervals (`stats_writers_summary.csv`).

G Reproducing the results

The repository's `README.md` gives every command. In short:

```
conda env create -f environment.yml
conda activate cos30049-a2
python -m pytest -q                      # the test suite
python -m explorer.cli all --offline     # every DS1 result
python -m explorer.cli figures
python tools/compare_tables.py           # rebuild against the committed tables
```

`DATA_CARD.md` documents every dataset, file and column, and `MODEL_CARD.md` every final model.